

Xarxes

PRÀCTICA 5

Mario VILAR

20392724

David Díez

20392643

7 de gener de 2025

Professors: Manel López i Ferran Adelantado



UNIVERSITAT DE
BARCELONA

ÍNDEX

1	Introducció	2
2	Qüestions teòriques	3
3	Conclusions	20

INTRODUCCIÓ

L'Organització internacional per a la Estandardització (ISO) i la Comissió Electrotècnica internacional (IEC) van ser els encarregats de definir les normes estàndards de comunicació entre equips de forma i manera que la transmissió i recepció de dades fos independent del sistema físic i de l'aplicació lògica utilitzada pels usuaris. Fixem-nos per exemple en una simple connexió a Internet. Deixant a banda l'àmplia quantitat de sistemes hardware que es poden fer servir (ordinadors de sobretaula, portàtils, PDAs, mòbils intel·ligents, tablets...) i sistemes operatius existents (Windows x, Linux, MAC OS, Android, Unix, ...) un usuari final pot fer servir per connectar-se a internet diversos navegadors combinats amb qualsevol de les anteriors característiques, així podem connectar-nos a través de l'Internet Explorer, Mozilla Firefox, Google Chrome i Safari entre d'altres. El que està clar és que tots els equips han de fer servir el mateix estàndard. Així, tots implementaran com a capa d'aplicació el protocol HTTP, a la capa de transport utilitzaran UDP, a la capa de xarxa faran servir IP i a la capa d'accés al medi el sistema que estigui implementat (Ethernet, Wifi, ATM, ...)

La principal missió del model OSI és permetre la connexió entre usuaris independentment de la plataforma utilitzada. Per tal d'aconseguir-ho, el model s'estructura en 7 capes diferents, cadascuna amb una funció específica, necessària per relacionar-se amb els altres sistemes. Cada capa es sustenta en la capa immediatament inferior, la qual realitzarà una sèrie de funcions més primitives que quedaran encapsulades i per tant seran transparents per les capes superiors. Aquesta encapsulació de les dades associades a la pròpia capa s'anomena "Unitat de Dades de Protocol" o PDU. La característica de la PDU, és que es tracta d'un format estàndard que poden interpretar totes les capes que implementen el mateix protocol. És per això que de vegades es parla de una comunicació virtual entre parells de capes, on el que s'intercanvien són les PDUs associades a aquestes. Per aquesta raó es veu, a la figura 1, una línia que uneix les capes de nivell N origen amb la capa de nivell N destí, intercanviant-se de forma virtual, la PDU corresponent. El mateix passa per la capa $N + 1$. Ara bé, la informació realment passa de una capa a la immediatament inferior o superior. En el cas del node origen, la transferència es produeix des de l'Aplicació fins a la capa física, on les dades s'envien a través del medi. Per fer aquest procés, La PDU del nivell $N + 1$ s'enviarà a través de les primitives d'intercanvi existents entre la capa $N + 1$ i la capa N . La informació d'intercanvi es guarda en una capçalera on tindrem la Informació de Control de la Interfície (ICI), que s'afegeix a la PDU formant una IDU o Unitat de Dades de Interfície. A la capa N , es rep la IDU, s'elimina la capçalera d'intercanvi (ICI) i es queda amb la $N + 1$ PDU, que a la capa N es coneix com la Unitat de Dades de Servei, o SDU de la capa N . És a dir, el que és la $N + 1$ -PDU passa a ser la N -SDU. A la capa N , ara introduïrem la informació necessària per que la informació pugui ser extreta per la capa N del destí. Això implica introduir la capçalera de la capa N . Per tant, la N -SDU més la capçalera associada a la capa N dona N -PDU.

Cada capa té el mateix protocol que la seva germana en els altres dispositius. L'intercanvi d'informació

entre capes germanes és el PDU. Per altra banda, la pròpia estructura de la comunicació fa que el moviment físic de les dades vagin d'una capa a les immediatament superiors o inferiors. En aquest sentit, s'introdueix el concepte de SDU i encapsulació.

Tot i aquesta extrema definició del model OSI, que té en consideració tots els detalls necessaris en una comunicació, aquest model no es va arribar a implementar en la seva totalitat mai. La aparició de TCP/IP, protocol previ fins i tot a la definició del model OSI, es va convertir en l'arquitectura de comunicacions dominant en el mercat comercial. Com s'ha dit, TCP/IP es va especificar i es va començar a utilitzar de forma generalitzada abans de la normalització ISO, per aquesta raó, la majoria d'institucions amb necessitats d'intercanvi d'informació van fer servir el protocol TCP/IP donada la seva disponibilitat immediata. Finalment, potser el punt clau, la xarxa Internet està basada en TCP/IP.

L'arquitectura TCP/IP consta bàsicament de 4 capes, que compacten notablement les funcionalitats que es mostraven en el model OSI. TCP/IP pren el nom de dos dels protocols que conformen aquest model, el protocol TCP, que pertany a la capa de transport o extrem a extrem i el protocol IP que seria la capa de xarxa del model OSI però amb menys funcionalitat. La figura 3 mostra l'estructura de capes així com la seva funcionalitat.

En aquesta pràctica s'analitzaran els conceptes de xarxes a través de capturar, filtrar i interpretar paquets utilitzant Wireshark, així com la simulació de xarxes amb Cisco Packet Tracer. Es revisaran aspectes com la comunicació TCP/IP, el model OSI i protocols com DNS, ICMP, HTTP i NAT.

2

QÜESTIONS TEÒRIQUES

Exercici 2.1. Doneu al botó d'inici (start). Veureu que apareix tot el transit que s'està monitoritzant a la xarxa local a la qual esteu treballant. Aquesta quantitat d'informació és tant elevada que resulta complicat estudiar-la en detall. Anem a filtrar les dades que ens arriben. El primer que farem serà un filtratge per IP. Per això farem servir la nostra IP, que havíem trobat prèviament en fer el ipconfig. Poseu el següent a la zona de filtratge que trobareu just a sota de les icones:

```
ip.addr eq <la vostra IP>
```

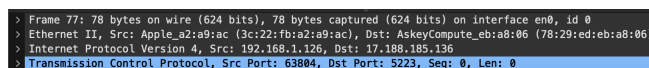
Escolliu de la llista una de les captures on el protocol sigui TCP i seleccioneu-la. A la finestra horitzontal intermitja, es desglossa tot el paquet seleccionat. A la finestra inferior es desglossa el que teniu seleccionat a la finestra intermitja. Així, si cliqueu a Ethernet apareix la MAC vostra i la del destinatari.

1. Com es descriu la vostra adreça MAC?
2. A la adreça MAC hi han dues parts clarament diferenciades. A que corresponen?
3. Compara el que apareix amb el que surt amb un ipconfig/all.
4. Repassa els diferents camps que apareixen a la capçalera IP i amb l'ajut dels llibres i/o Internet identifica

que fa cada un dels camps.

Resolució. Contestem una per una cada pregunta:

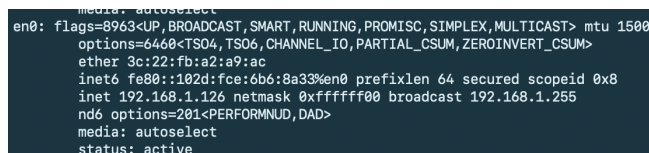
1. La adreça MAC es descriu com una seqüència de 48 bits representats en hexadecimal i separats per dos punts (per exemple, XX:XX:XX:XX:XX:XX). Aquesta adreça la podem obtenir utilitzant `ipconfig /all` directament a Power Shell o bé a través de Wireshark mirant a la pantalla inferior dreta.



```
> Frame 77: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0, id 0
> Ethernet II, Src: Apple_a2:a9:ac (3c:22:fb:a2:a9:ac), Dst: AskeyCompute_eb:a8:06 (78:29:ed:eb:a8:06)
> Internet Protocol Version 4, Src: 192.168.1.126, Dst: 172.16.0.105
> Transmission Control Protocol, Src Port: 53804, Dst Port: 5223, Seq: 0, Len: 0
```

Figura 1: Captura del paquet TCP i on podem veure les adreces MAC al costat d'Ethernet II.

2. Totes les adreces MAC estan dividides en dues parts:
 1. Els primers 24 bits corresponen a l'Identificador Únic d'Organització (OUI), que indica el fabricant de l'equip.
 2. Els darrers 24 bits representen un identificador únic assignat pel fabricant.
3. Òbviament l'adreça MAC depèn del dispositiu, no del lloc on la busquem i per tant el resultat obtingut a Wireshark és el mateix que utilitzant `ipconfig /all`. En cas de no coincidir podria indicar que s'està monitoritzant una interfície diferent de la seleccionada, però no és el cas:



```
media: autoselect
en0: flags=8963<UP,BROADCAST,SMART,RUNNING,PROMISC,SIMPLEX,MULTICAST> mtu 1500
options=6460<TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
ether 3c:22:fb:a2:a9:ac
inet6 fe80::102d:fce:6b6:8a33%en0 prefixlen 64 secured scopeid 0x8
inet 192.168.1.126 netmask 0xfffff00 broadcast 192.168.1.255
nd6 options=201<PERFORMNUD,DAD>
media: autoselect
status: active
```

Figura 2: Resultat del `ifconfig` al port en0, on el Mac es connecta per Wi-Fi.

4. Els principals camps de la capçalera IP són:
 1. **Version:** Indica la versió del protocol IP (IPv4 o IPv6).
 2. **IHL (Internet Header Length):** Longitud de la capçalera expressada en blocs de 4 bytes.
 3. **Type of Service (ToS):** Defineix la qualitat de servei per a la transmissió de dades.
 4. **Total Length:** Longitud total del paquet, incloent capçalera i dades.
 5. **Identification:** Identifica únicament cada fragment de paquet.
 6. **Flags:** Controlen la fragmentació dels paquets.
 7. **Fragment Offset:** Indica la posició del fragment dins del paquet original.
 8. **Time to Live (TTL):** Limita la vida del paquet per evitar bucles infinits.
 9. **Protocol:** Especifica el protocol de la capa superior (per exemple, TCP o UDP).
 10. **Header Checksum:** Verifica la integritat de la capçalera.
- II. **Source Address:** Adreça IP de l'origen.

12. **Destination Address:** Adreça IP del destinatari.
13. **Options:** Opcional, per a característiques addicionals. ■

Exercici 2.2.

1. *Feu una captura de les comunicacions. Un cop està llençada la captura, des de consola feu:*
`telnet time-A.timefreq.blrdoc.gov 13`
Aneu a Capture i cliqueu stop. Tenint en compte que no heu introduït una adreça IP, busqueu el protocol DNS que permet connectar-nos amb un servidor de DNS per tal de discernir quina IP correspon a la màquina time-A.timefreq.blrdoc.gov.
2. *Un cop determinada la IP, a quin port ens estem connectant? Quin protocol de transport fem servir?*
3. *Identifica l'intercanvi de comunicacions que es produeix a nivell de DNS. Quin protocol de transport fa servir DNS? Perquè? Quina és la IP del servidor de DNS? Com s'especifica la resposta? Què respon?*
4. *Feu el diagrama temporal descrivint detalladament l'intercanvi d'informació entre el vostre ordinador i el servidor de DNS.*
5. *Un cop coneguda la IP destí, proporcionada pel servidor de DNS, identifica l'intercanvi de control que es produeix a nivell de TCP per la transmissió de la informació. Expliqueu que fa cada paquet i feu un diagrama temporal on es representa aquest intercanvi. Pren molta rellevància la utilització dels flags a TCP. Indiqueu que fan i com es fan servir per gestionar la comunicació.*

Resolució.

1. El servei de noms de domini utilitza el protocol DNS per fer les consultes i les respostes. Es tracta d'un protocol de capa d'aplicació que pot utilitzar tant UDP com TCP en la capa de transport. La captura demanada és

No.	Time	Source	Destination	Protocol	Length	Info
10850	991.176361	142.250.200.100	192.168.1.12	QUIC	984	Protected Payload (KPO)
10851	991.177119	192.168.1.12	142.250.200.100	QUIC	120	Handshake, DCID=fe6f9911a76c7611
10852	991.177246	192.168.1.12	142.250.200.100	QUIC	73	Protected Payload (KPO), DCID=fe6f9911a76c7611
10853	991.182066	142.250.200.100	192.168.1.12	QUIC	67	Protected Payload (KPO)
10854	991.191613	142.250.200.100	192.168.1.12	QUIC	1292	Initial, SCID=fe6f9911a76c7611, PKN: 8, ACK, CRYPTO, PADDING
10855	991.191613	142.250.200.100	192.168.1.12	QUIC	162	Protected Payload (KPO)
10856	991.192194	192.168.1.12	142.250.200.100	QUIC	75	Protected Payload (KPO), DCID=fe6f9911a76c7611
10857	992.978492	142.250.201.74	192.168.1.12	UDP	122	443 → 63344 Len=80
10858	992.978492	142.250.201.74	192.168.1.12	UDP	122	443 → 63344 Len=80
10859	992.978786	192.168.1.12	142.250.201.74	UDP	76	63344 → 443 Len=34
10863	1000.726132	192.168.1.12	34.120.195.249	TCP	55	[TCP Keep-Alive] 22929 → 443 [ACK] Seq=1300 Ack=532 Win=510 Len=1
10864	1000.746276	34.120.195.249	192.168.1.12	TCP	66	[TCP Keep-Alive ACK] 443 → 22929 [ACK] Seq=532 Ack=1301 Win=1043 Len=0 SLE=1300 SRE=1301
10865	1000.828818	192.168.1.12	34.96.102.34	TCP	55	[TCP Keep-Alive] 22928 → 443 [ACK] Seq=1297 Ack=782 Win=509 Len=1
10866	1000.828818	34.96.102.34	192.168.1.12	TCP	66	[TCP Keep-Alive ACK] 443 → 22928 [ACK] Seq=782 Ack=1298 Win=1043 Len=0 SLE=1297 SRE=1298
10867	1000.897613	192.168.1.12	34.117.201.170	TCP	55	[TCP Keep-Alive] 22683 → 443 [ACK] Seq=481 Ack=458 Win=512 Len=1
10868	1000.913209	34.117.201.170	192.168.1.12	TCP	66	[TCP Keep-Alive ACK] 443 → 22683 [ACK] Seq=458 Ack=482 Win=1044 Len=0 SLE=481 SRE=482
10869	1002.397577	192.168.1.12	239.255.255.250	TGMPv2	46	Membership Report group 239.255.255.250
10870	1002.427809	192.168.1.12	34.120.52.64	TCP	55	[TCP Keep-Alive] 22870 → 443 [ACK] Seq=11767 Ack=892670 Win=4103 Len=1
10871	1002.447913	34.120.52.64	192.168.1.12	TCP	66	[TCP Keep-Alive ACK] 443 → 22870 [ACK] Seq=892670 Ack=11768 Win=1010 Len=0 SLE=11767 SRE=11768
10873	1003.569572	192.168.1.12	142.250.184.3	TCP	55	[TCP Keep-Alive] 23036 → 443 [ACK] Seq=1799 Ack=6476 Win=130048 Len=1
10879	1007.980827	192.168.1.12	142.250.201.74	UDP	71	63344 → 443 Len=29
10880	1008.001352	142.250.201.74	192.168.1.12	UDP	68	443 → 63344 Len=26

on podem veure el trànsit que està analitzant Wireshark.

Un cop hem introduït la comanda `telnet time-A.timefreq.blrdoc.gov 13` al Power Shell tornem a Wireshark i filtrem per dns (això és, filtrant el protocol a la barra superior). Ara podem veure que les últimes dues comunicacions han sigut l'enviament i recepció de la petició, respectivament.

Si mirem la columna de Info que ens dona Wireshark veiem que la primera comunicació és

```
1 Standard query 0x80df A time-A.timefreq.bldrdoc.gov
```

i la segona

```
1 Standard query response 0x80df A time-A.timefreq.bldrdoc.gov CNAME time-a-b.nist.gov A
  ↪ 132.163.96.1
```

Això ens fa pensar que la IP amb la que ens estem comunicant és la 132.163.96.1.

2. Si anem a la query (no a la query response) podem veure que Wireshark ens diu

```
1 User Datagram Protocol, Src Port: 49666, Dst Port: 53
2 Source Port: 49666
3 Destination Port: 53
4 Length: 53
5 Checksum: 0x83a4 [unverified]
6 [Checksum Status: Unverified]
7 [Stream index: 129]
8 [Stream Packet Number: 17]
9 [Timestamps]
10 UDP payload (45 bytes)
```

i per tant veiem que ens estem connectant al port 49666 (aquest port canviarà en funció del dispositiu, no el 53 que és específic de DNS). Mirant la línia 10 veiem que estem utilitzem el protocol UDP¹.

3. Com ja hem explicat a la primera pregunta, nosaltres enviem un missatge a la IP pública 132.163.96.1 que després rebem de tornada. Usualment, tant les consultes del client com les respostes del servidor es poden encabir en un datagrama (512 bytes) i s'utilitza UDP (de fet, generalment es diu que el DNS usa UDP). Però si la informació a transmetre és àmplia (per exemple, una resposta amb una llista amb molta informació), la comunicació es passa a TCP automàticament. Un altre cas en què la informació és TCP és quan es realitza la transferència d'informació d'una zona entre servidors primaris i secundaris. El servidor DNS utilitza el port privilegiat 53. Recordem, a més, que es tracta d'un protocol de capa d'aplicació. Si mirem la informació detallada de la comunicació de tornada (la *response*) amb Telnet veiem que diu

```
1 Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.12
2 Protocol: UDP (17)
3 Source Address: 192.168.1.1
```

¹ UDP (User Datagram Protocol) és un protocol del nivell de transport del model OSI, basat en l'intercanvi de datagrames. UDP permet l'enviament de datagrames a través d'una xarxa sense que s'hagi establert prèviament una connexió, ja que el mateix datagrama incorpora suficient informació d'adreçament a la seva capçalera.

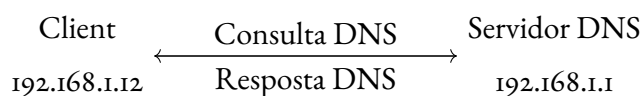
```
4 Destination Address: 192.168.1.12
5 [Stream index: 3]
```

Això vol dir que el la IP de la DNS és 192 . 168 . 1 . 1, que correspon a la IP del router (serà el router qui s'encarregui de traslladar les peticions DNS als servidors DNS que tingui declarats internament). Companyies com Movistar tenen unes adreces DNS dedicades per als seus clients. En el meu cas (Mario), l'adreça d'aquesta DNS seria 80 . 58 . 61 . 250.

4. El procés típic consta dels següents passos:

1. **Resolució de noms (DNS Query):** L'ordinador vol accedir a un domini (com el de l'exercici anterior, per exemple) i envia una consulta DNS al servidor DNS configurat a la seva xarxa, utilitzant el protocol UDP (port 53).
2. *Resposta del servidor DNS:* El servidor DNS cerca al seu registre i respon amb l'adreça IP associada al domini. Si no la troba, pot fer una consulta recursiva a altres servidors DNS.
3. *Finalització de la resolució:* L'ordinador rep la resposta amb l'adreça IP corresponent, la desa a la seva memòria cau (cache) per consultes futures i inicia la connexió amb aquesta IP.

Un possible esquema seria el següent:



5. El TCP dona fiabilitat a l'aplicació, és a dir, garanteix el lliurament de tota la informació en el mateix ordre en què ha estat transmesa per l'aplicació d'origen. Per aconseguir aquesta fiabilitat, el TCP proporciona un servei orientat a la connexió amb un control de flux i errors.

TCP, a diferència d'UDP, intenta generar segments d'una mida òptima que s'anomena Maximum Segment Size (MSS), generalment el més gran possible per minimitzar el sobrecost (overhead en anglès) de les capçaleres, sense que produeixi fragmentació al nivell IP.

Proporciona fiabilitat mitjançant els següents controls:

1. Control d'errors. TCP no descarta segments posteriors quan arriben fora de seqüència. Permet les retransmissions selectives.
2. Control de flux (finestra advertida). Serveix per adaptar la velocitat entre l'emissor i el receptor. Vigila que l'emissor no envii els segments a més velocitat de la que pot fer servir el receptor per processar-los, de manera que es puguin perdre paquets per saturació de la memòria intermèdia de recepció.
3. Control de la congestió (finestra de congestió). Per adaptar la velocitat de l'emissor als encaminadors intermedis de la xarxa i evitar així que es col·lapsin les memòries intermèdies i es puguin perdre paquets.

A més:

- En transmissió, TCP s'encarrega de fragmentar les dades del nivell aplicació i els assigna un número de seqüència abans d'enviar els fragments al nivell IP.
- En recepció, com que els segments poden arribar fora d'ordre, TCP els ha de reordenar abans de passar-los als nivells superiors.

Podem deduir l'intercanvi de control que es produeix a nivell de TCP gràcies a la capçalera. Els seus camps són els següents:

- **Port Origen / Port Destí:** Identifiquen les aplicacions d'origen i destí.
- **Número de Seqüència:** Indica el primer byte d'aquest segment dins de la seqüència de bytes enviats fins al moment.
- **Número Inicial de Seqüència (ISN):** Primer número de seqüència escollit pel protocol TCP.
- **Número de Seqüència:** A partir de l'ISN. Per calcular els bytes enviats: $\text{Seq. Number} - \text{ISN}$.
- **Número d'Acknowledgment:** Conté el següent número de seqüència que el transmissor de l'ACK espera rebre ($\text{Seq. Number} + 1$ del darrer byte rebut correctament).
 - TCP és *full-duplex*: cada extrem manté el seu Seq. Number i Ack Number.
- **Longitud de la Capçalera:** Longitud total de la capçalera TCP en paraules de 32 bits.
 - Mida mínima: 20 bytes ($\text{header length} = 5$).
 - Mida màxima: 60 bytes ($\text{header length} = 15$).
- **Reserved:** Bits reservats per possibles ampliacions. Sempre es posen a 0.
- **Flags:** Hi ha 6 flags (bits) a la capçalera. Són vàlids quan estan a 1.
 - **URG (Urgent):** Indica que es fa servir el camp Urgent Pointer.
 - **ACK (Acknowledgment):** Indica que es fa servir el camp Acknowledgment Number. És vàlid per a tot excepte el SYN inicial.
 - **PSH (Push):** Indica que el receptor ha de passar les dades de la memòria intermèdia als nivells superiors tan ràpid com sigui possible.
 - **RST (Reset):** Activa aquest flag per interrompre una connexió.
 - **SYN (Synchronize):** S'utilitza per establir la connexió i sincronitzar els números de seqüència.
 - **FIN (Finalize):** Indica la finalització de la connexió.
- **Advertised Window Size:** Mida de la finestra que el receptor informa al transmissor per al control de flux. La mida màxima és de 65.535 bytes.
- **Checksum TCP:** Detecta errors en el segment TCP i assegura que no ha arribat a un destí incorrecte.
 - Igual que a UDP, el *checksum* es calcula amb una pseudo-capçalera + capçalera TCP + dades

TCP.

- La pseudo-capçalera inclou camps de la capçalera IP i la mida del segment TCP, però no es transmet.
- El càlcul del *checksum* és obligatori a TCP.
- **Urgent Pointer:** Vàlid si el flag URG = 1. Indica dades urgents dins del camp de dades. Es fa servir en casos excepcionals, com en interrupcions (Ctrl+C).
- **Options:** TCP permet afegir opcions a la capçalera.
 - **Maximum Segment Size (MSS):** S'utilitza durant l'establiment de la connexió per definir el MSS. Per exemple, si la xarxa és Ethernet (MTU = 1500), aleshores MSS = 1460.
 - **Window Scale Factor:** Amplia el valor de la finestra advertida per controlar fluxos més grans.
 - **Timestamp:** S'utilitza per al càlcul del RTT.
 - **SACK (Selective Acknowledgment):** Permet al receptor indicar blocs de segments correctes més enllà del confirmat pel camp ACK.
- **Padding:** Bytes afegits per assegurar que la capçalera tingui una mida múltiple de 32 bits.

Els 6 flags que acabem d'explicar són bits de control que permeten gestionar l'establiment, manteniment i finalització de connexions TCP. Aquests flags s'utilitzen per coordinar les accions entre els dos extrems de la connexió i per indicar estats o situacions especials durant la transmissió de dades. Per exemple:

- Establiment de connexió. El client envia un SYN per començar la connexió, el servidor respon amb un SYN-ACK i el client confirma amb un ACK.
- Transmissió de dades. Es fan servir ACK per confirmar la recepció de dades i PSH si cal passar-les immediatament a l'aplicació.
- Finalització de la connexió. Es fa servir FIN per tancar la connexió de manera ordenada, i RST si cal interrompre-la de forma brusca. ■

Exercici 2.3.

1. Des de capture, cliqueu a start i feu un ping a una adreça pública coneguda d'Internet (per exemple *www.google.com*). Captureu el transit de la comunicació i desglosseu la comunicació en sí. Expliqueu detalladament la captura, tal i com s'ha fet en l'exercici anterior.
2. Què és el protocol ICMP? Com funciona aquest protocol? Quins identificadors i flags fa servir?
3. Obriu el navegador i poseu `http://ip_obtinguda_a_través_del_ping`. S'obre la pàgina? Que captura el sniffer? Feu una explicació detallada.
4. Desglosseu la captura per connectar amb la web. Aneu al protocol TCP. Quin port de sortida heu fet servir? Identifiqueu algun protocol de control de flux? Expliqueu detalladament el que heu capturat.

Resolució.

1. La captura demanada és la següent

7722	472.497261	185.93.182.137	192.168.1.12	WireGu...	141	Transport Data, receiver=0xEE1D6639, counter=1397, datalen=67
7723	474.592181	192.168.1.12	142.250.200.100	ICMP	74	Echo (ping) request id=0x0001, seq=333/19713, ttl=128 (reply in 7724)
7724	474.689224	142.250.200.100	192.168.1.12	ICMP	74	Echo (ping) reply id=0x0001, seq=333/19713, ttl=117 (request in 7723)
7725	475.610305	192.168.1.12	142.250.200.100	ICMP	74	Echo (ping) request id=0x0001, seq=334/19969, ttl=128 (reply in 7726)
7726	475.627329	142.250.200.100	192.168.1.12	ICMP	74	Echo (ping) reply id=0x0001, seq=334/19969, ttl=117 (request in 7725)
7727	475.668015	185.93.182.137	192.168.1.12	WireGu...	189	Transport Data, receiver=0xEE1D6639, counter=1398, datalen=115

on podem veure a la columna *info* que s'indica que està fent el ping corresponent. En aquest cas la direcció ip de Google és la 142.250.200.100 i ho podem saber pel valor a la columna *destination* quan fem *request* i el valor a la columna *source* quan obtenim el *reply*. Un altra dada interessant és la del TTL. Veiem que en les comunicacions de *reply* el valor de TTL val 117. Aquest valor coincideix amb el que hi ha a la consola quan fem el ping.

```
Haciendo ping a www.google.com [142.250.200.100] con 32 bytes de datos:
Respuesta desde 142.250.200.100: bytes=32 tiempo=17ms TTL=117
Respuesta desde 142.250.200.100: bytes=32 tiempo=17ms TTL=117
Respuesta desde 142.250.200.100: bytes=32 tiempo=16ms TTL=117
Respuesta desde 142.250.200.100: bytes=32 tiempo=16ms TTL=117

Estadísticas de ping para 142.250.200.100:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 16ms, Máximo = 17ms, Media = 16ms
```

Si volem veure la informació detallada del que obtenim podem fer clic sobre una comunicació capturada i si despleguem la secció on diu *Internet Control Message Protocol* trobem això:

```
▼ Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x4c0e [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 333 (0x014d)
  Sequence Number (LE): 19713 (0x4d01)
  [Response frame: 7724]
  ▼ Data (32 bytes)
    Data: 61626364656666768696a6b6c6d6e6f707172737475767776162636465666676869
    [Length: 32]
```

De forma esquemàtica això ens fa veure que les dades que s'envien són

1. Un camp de **tipus** que especifica si el paquet és una petició (request, type 8) o una resposta (reply, type 0).
 2. Un **codi** que proporciona informació addicional sobre el tipus.
 3. Un **checksum** per verificar la integritat del paquet.
 4. **Identificador** i **Seqüència**, que es fan servir per relacionar les peticions amb les respostes.
 5. Un camp de **dades** que conté informació de prova.
2. El protocol ICMP (Internet Control Message Protocol) s'utilitza per enviar missatges d'error i informació operativa entre dispositius en una xarxa IP. Funciona enviant paquets de prova entre dos dispositius, comprovant així la connectivitat i mesurant el temps de resposta.

Els identificadors clau inclouen:

1. **Tipus i Codi:** Indiquen el tipus de missatge (per exemple, petició o resposta).
2. **Checksum:** Verifica la integritat del missatge.
3. **Identificador i Seqüència:** Asseguren que la resposta correspon a la petició enviada.

Recomanem veure l'última captura de l'exercici anterior ja que hi ha una visió més pràctica i real del que està passant.

3. Fent ping a Google obtenim la direcció 142.250.200.100 i fent `http://142.250.200.100/` sobre la pàgina web associada sense problemes. El sniffer captura la següent informació:

1. Una comunicació TCP establint una connexió amb el servidor web a través del port 80 (HTTP).
2. Els paquets contindran:
 1. **Petició HTTP:** Enviada des del navegador al servidor (GET, POST, etc.).
 2. **Resposta HTTP:** Resposta del servidor amb el contingut sol·licitat.

4. El port de sortida és un **port efímer**, seleccionat automàticament per la màquina client (normalment entre 49152 i 65535). En el nostre cas, posem `ip.addr eq 142.250.200.100` i trobem el paquet enviat a través de la capa de transport (protocol TCP) corresponent. Depenent de si triem el paquet d'anada o de tornada, els ports de sortida s'intercanvien. Suposant que prenem de source el nostre ordinador, tindríem que el port de sortida és el 58424, i el d'arribada el 80 (el port HTTP).

Pel que fa als mecanismes de control de flux, el protocol TCP utilitza els següents:

1. **ACK (Acknowledgement):** Indica al remitent que pot enviar el següent paquet, una vegada el receptor ja ha verificat les dades.
2. **Sliding window:** Permet ajustar la quantitat de dades enviades en funció de la capacitat del destinatari (és a dir, la quantitat de bytes que el dispositiu receptor pot acceptar i processar simultàniament). Aquest valor es defineix durant la connexió inicial i determina el límit de dades que l'emissor pot enviar abans de rebre un ACK. Si hi ha retards en l'ACK, l'emissor pausa l'enviament de dades, ajustant així la velocitat de transmissió segons l'estat de la xarxa i del receptor.

En el cas de la captura següent, el que veiem és que es mostra un valor de 512 bytes, que és la quantitat màxima de dades que el receptor pot acceptar i processar sense haver d'enviar un acús de rebut (ACK). També es calcula el window size escalat com a 131072 bytes, tenint en compte un factor d'escalat (scaling factor) de 256. Això permet gestionar sessions amb volums més grans de dades. Només pot enviar més dades quan rebí un nou ACK que confirmi que el receptor ha processat els bytes enviats anteriorment. El valor del camp Flags: 0x0010 (ACK) confirma que aquest paquet és un acús de rebut. Això és típic en el procés de control de flux, ja que s'informa al transmissor que pot continuar enviant dades.

En la captura següent es poden observar els següents passos:

1. Establiment de connexió (s'activen **SYN**, **SYN-ACK**, **ACK**, que a la pràctica corresponen a una sèrie de bits a la part del flag de la trama a nivell TCP).
2. Enviament de la petició HTTP (GET).
3. Recepció de la resposta HTTP amb el contingut de la pàgina.
4. Finalització de la connexió (**FIN**, **ACK**). ■

Adjuntem una captura de pantalla d'això que acabem d'explicar:

```

Transmission Control Protocol, Src Port: 58424, Dst Port: 80, Seq: 2, Ack: 2,
  Source Port: 58424
  Destination Port: 80
  [Stream index: 124]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 2      (relative sequence number)
  Sequence Number (raw): 3587097140
  [Next Sequence Number: 2      (relative sequence number)]
  Acknowledgment Number: 2      (relative ack number)
  Acknowledgment number (raw): 928722511
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x010 (ACK)
  Window: 512
  [Calculated window size: 131072]
  [Window size scaling factor: 256]
  Checksum: 0x192e [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]

```

Exercici 2.4. Obre el programa simulador Cisco Packet Tracer i crea dues xarxes connectades a través del núvol. Cada una de les xarxes ha de tenir un router genèric i dos switchos de 24 canals. Connecta uns quants ordinadors (entre cinc i vuit) als switchos. En una de les xarxes munta un subnetting variable i assigna les IP's de manera estàtica. A l'altra xarxa munta en el router un servidor DHCP per tal d'assignar IPs de forma dinàmica. En totes dues xarxes configura NAT.

1. Fes PING des de un dels ordinadors de la xarxa dinàmica a un dels ordinadors de l'altra xarxa. Mostra els diferents empaquetats que es realitzen.
2. Configura una pàgina web en un dels equips i connectat des de l'altra xarxa a aquesta pàgina. Mostra els diferents empaquetats que es realitzen.
3. Mostra una imatge on es visualitzi la xarxa. Captura el tràfic generat.

Resolució. No entrarem en molt detall pel que fa a l'explicació de com muntem les xarxes, ja que s'ha treballat extensament en les dues pràctiques anteriors. Alguns punts a destacar:

1. Al Routero muntem el DHCP. Caldrà definir un *default gateway*, un servidor DNS, la xarxa on treballarem i si exclouem algunes IPs de l'assignació dinàmica (no serà el cas).
2. La connexió al núvol es farà exactament com a la pràctica anterior. Per fer-ho, usarem la xarxa pública que se'ns proposava a la pràctica anterior. En aquest cas, però, no ens haurem de preocupar per

un tercer router que complicava lleugerament l'establiment del protocol Frame Relay. Els serials dels routers s'hauran d'encapsular en aquest protocol i haurem d'omplir les taules d'encaminament estàtic adequadament.

3. La **màscara de subxarxa** de longitud variable (VLSM) és una estratègia de subnetting on totes les màscares de subxarxa poden tenir mides diferents. A diferència del FLSM, aquest procés ens permet utilitzar diverses màscares per a diferents subxarxes d'una mateixa xarxa de classe A, B o C. Amb VLSM, un espai d'adreces IP es pot dividir en una jerarquia ben definida de subxarxes amb diferents mides. Malauradament dos switchos són pocs per demostrar els avantatges del subnetting variable. Tot i així, la idea seria fer el següent al Router1:

	hosts disponibles	màscara	xarxa	primera IP	última IP	broadcast
1	$2^7 - 2 = 126$	255.255.255.128	192.168.2.0/25	192.168.2.1	192.168.2.126	192.168.2.127
2	$2^4 - 2 = 14$	255.255.255.240	192.168.2.128/28	192.168.2.129	192.168.2.142	192.168.2.143

Taula 1: Taula resum del subnetting que farem.

El Gateway de les subnets seran les respectives primeres IPs. Un exemple de configuració de les interfícies del router seria el següent:

```

1 Router(config)# int Fa0/0
2 Router(config-if)# ip address 192.168.2.1 255.255.255.128
3 Router(config-if)# no shutdown
4 Router(config-if)# exit
5 Router(config)# write memory

```

Arribats a aquest punt, assignaríem estàticament les IPs dels diferents PCs. El DNS pot ser 8.8.8.8, per exemple.

4. Els problemes que vam tenir amb la NAT a la pràctica anterior seguiran apareixent. Optarem per veure com funciona sense i finalment l'afegirem perquè ens ho demana l'enunciat.

Configuraríem la xarxa com hem anat detallant, de moment sense NAT. Si volem, podem comprovar que cada xarxa per separat està ben configurada abans de connectar-la al núvol. Una vegada les dues xarxes funcionen i configurem els ports serial, les taules d'enrutament i el Frame Relay, podem veure l'esquema final. Utilitzem dos sniffers per veure el tràfic tant sortint com entrant en una de les xarxes; en particular, ens interessen els empaquetats que es realitzen.

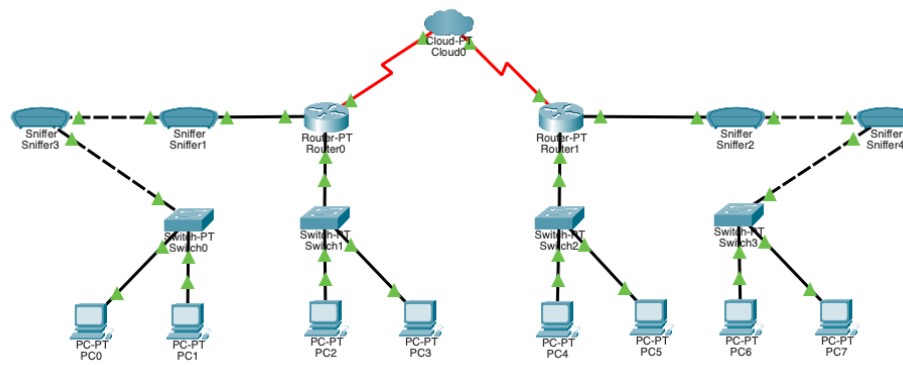


Figura 3: Esquema final de la nostra xarxa.

El resultat seria que podem fer pings des de la xarxa dinàmica a la xarxa amb subnetting variable:

Vis.	Time(sec)	Last Device	At Device	Type
	0.000	--	PC1	ICMP
	0.001	PC1	Switch0	ICMP
	0.002	Switch0	Router0	ICMP
	0.003	Router0	Cloud0	ICMP
	0.004	Cloud0	Router1	ICMP
	0.005	Router1	Switch2	ICMP
	0.006	Switch2	PC4	ICMP
	0.007	PC4	Switch2	ICMP
	0.008	Switch2	Router1	ICMP
	0.009	Router1	Cloud0	ICMP
	0.010	Cloud0	Router0	ICMP
	0.011	Router0	Switch0	ICMP
	0.012	Switch0	PC1	ICMP

Reset Simulation ☒ Constant Delay Captured to: 0.012 s

Play Controls

Event List Filters - Visible Events

ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPSec, ISAKMP, IoT, IoT TCP, LACP, LLDP, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAP, POP3, PPP, PPPoE, PTP, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TAGACS, TCP, TFTP, Telnet, UDP, USB, VTP

Edit Filters Show All/None

Figura 4: Ping des de la xarxa dinàmica a la xarxa amb subnetting variable.

L'ordinador origen genera una petició ICMP Echo Request (capa 7, aplicació). Aquesta es fragmenta en un paquet IP (capa 3, xarxa). El paquet IP s'encapsula dins d'una trama Ethernet (capa 2, enllaç de dades). Finalment, es transmet com senyals elèctrics/òptics (capa 1, física). Si anem als sniffers, trobem que els empaquetats serien:

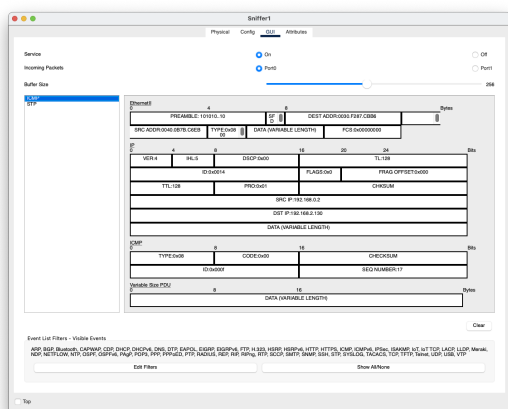


Figura 5: Empaquetat ICMP.

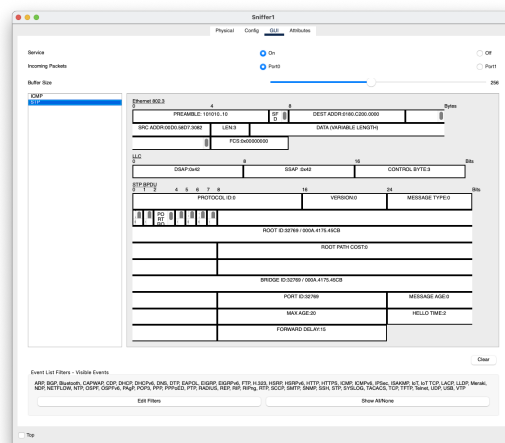


Figura 6: Empaquetat STP.

Aquest STP correspondria a un paquet llençat per un switch per evitar bucles (recordem que STP és un protocol de xarxa que s'utilitza per evitar bucles en una xarxa de switches). Si disposem dels sniffers que capturen el tràfic de la xarxa, i deixem passar un parell de minuts en *Realtime* veiem què han capturat:

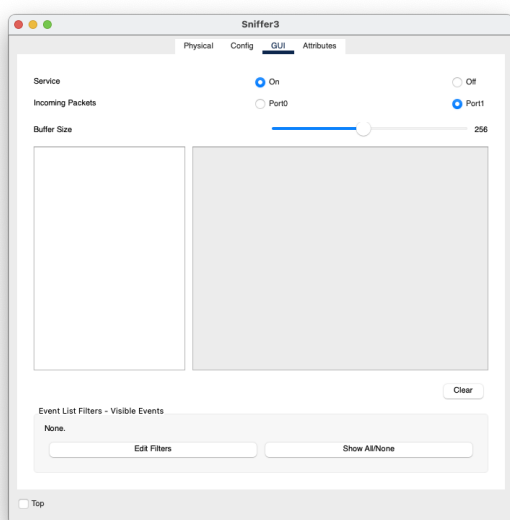


Figura 7: Sniffer que analitza el tràfic de baixada («de fora a dins»).

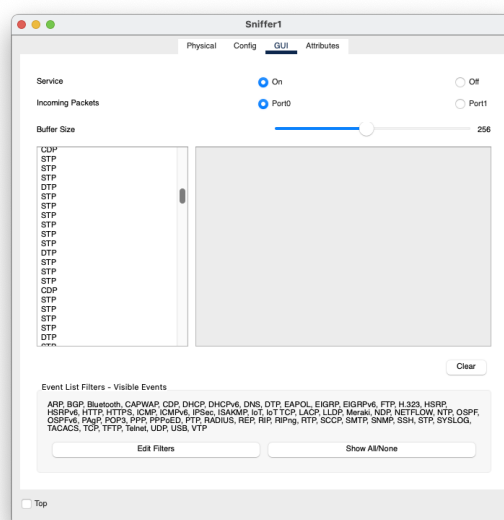


Figura 8: Sniffer que analitza el tràfic de pujada («de dins a fora»).

Correspon als dos que tenim a la branca de l'esquerra de la figura 3, els de la xarxa dinàmica. Com veiem, tot el tràfic es produeix de nivell local a nivell públic. Veiem generalment tres tipus de paquets, STP, CDP i DTP: STP (Spanning Tree Protocol), CDP (Cisco Discovery Protocol) i DTP (Dynamic Trunking Protocol) són protocols de capa 2 utilitzats per gestionar switches i connexions LAN, treballant amb trames Ethernet i sense dependre de les IPs. En canvi, ICMP opera a la capa 3.

Ara passem al mode Simulation i pengem un servidor des de la xarxa amb subnetting i li assignem la IP 192.168.2.132. Fem les configuracions necessàries i visitem amb el navegador del PC1: `http://192.168.2.132`, que visualitzarà el fitxer `index.html` que té emmagatzemat el servidor.



Figura 9: Fitxer html del servidor visitat des del PC1.

Si anem als sniffers, trobem que els paquets serien iguals que els que havíem trobat al Wireshark. Mostrats de manera diferent, però amb la mateixa informació:

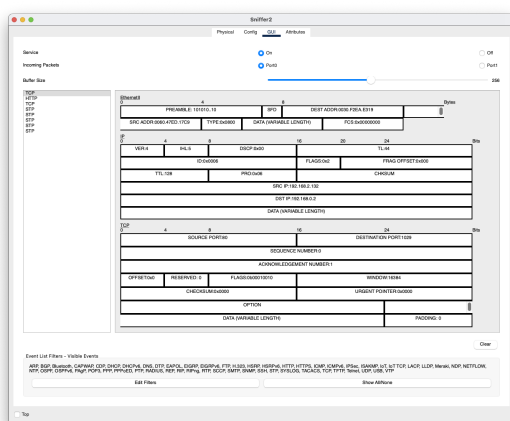


Figura 10: TCP.

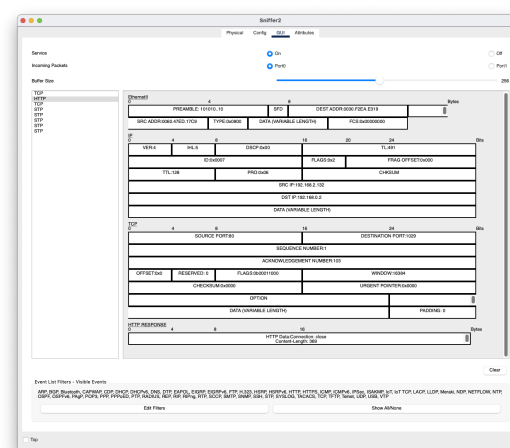


Figura 11: HTTP.

La primera imatge mostra un paquet amb una estructura Ethernet-II que especifica adreces MAC d'origen i destí, i un camp TYPE que indica que és un paquet IP (0x0800). Al nivell IP, el camp TTL és 128 i el protocol especificat és TCP (PRO: 0x06), i podem trobar *les adreces d'origen i destí*. El camp de dades té una longitud variable, en funció del que s'hagi de transportar. El paquet TCP mostra un port d'origen 80 (HTTP) i un port de destí 1029, amb el nombre de seqüència 0 i una confirmació (ACK) del número 1. Aquest últim

hauria d'estar a zero al principi (si està a 1 suggeriria que és part d'una comunicació establerta), però la captura de pantalla no és del primer paquet; aquests bits segueixen una certa lògica que explicarem més endavant.

La segona imatge és similar, però amb contingut específic d'una resposta HTTP. Al nivell IP tornem a trobar les IPs d'origen i destí², així com els flags, el TYPE i tot allò que ja hem comentat. Al nivell TCP, el número de seqüència és 1, i l'ACK és 103, indicant la continuació d'un flux de dades. Sota el camp de dades, s'observa una resposta HTTP que inclou capçaleres com «Connection: close» i «Content-Length: 359». Això indica que és un paquet que transporta dades de contingut HTTP. Com hem dit abans, acabarem de detallar el procés més endavant.

Al Wireshark, com hem comprovat a la primera part d'aquesta pràctica, també es poden veure les capes Ethernet-II, IP i TCP, amb camps com les adreces MAC, IP d'origen i destí, ports, nombres de seqüència, etc. Igual que aquí, els paquets TCP mostraran les fases de connexió (SYN, ACK) i, quan es tracta de protocols d'aplicació com HTTP, es podrà veure el contingut de la comunicació, capçaleres, dades...

La ruta que han seguit els paquets es pot consultar amb l'Event List:

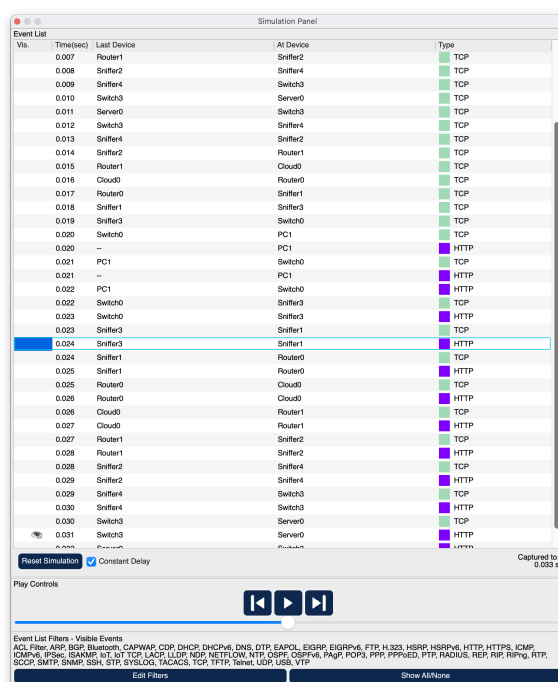


Figura 12: Event list.

Quan s'envia un paquet HTTP, aquest necessita una connexió establerta prèviament a través de TCP. Això és perquè HTTP és un protocol d'aplicació que es basa en TCP per assegurar el lliurament fiable de les dades. Després d'establir la connexió TCP (el que seria el primer paquet TCP, fins al segon 0.020), s'envia un nou paquet TCP juntament amb el paquet HTTP que conté el propi contingut del fitxer emmagatzemat al

² En aquest cas tenim una *resposta*, pel que l'adreça d'origen és el servidor que conté el fitxer html que s'està consultant, i el destinatari és l'ordinador que ha fet la sol·licitud (request).

servidor en format html.

1. *Primer paquet TCP*: S'envia des de PC1 al servidor per establir la connexió.

2. *Resposta del servidor*:

- El Acknowledgement Number (ACK) canvia a 1.
- El flag canvia de 0b00000010 (SYN) a 0b00010010 (SYN + ACK).
- El paquet torna a PC1.

3. *Segon paquet TCP + HTTP*: ACK = 1, Sequence Number = 1 i amb flag igual a 0b00010000 (PSH: dades enviades immediatament).

- S'envia al servidor, que respon amb ACK = 103 i flag = 0b00011000 (PSH + ACK)
- El paquet torna a PC1 i la comunicació finalitza.

I ja per acabar aquesta qüestió, la clau aquí seria veure **com els paquets canvien quan van cap al núvol: abans es transporten sobre Ethernet II i després, sobre Frame Relay**. És a dir, són d'una manera abans i després del router. Per exemple, sobre el paquet HTTP:

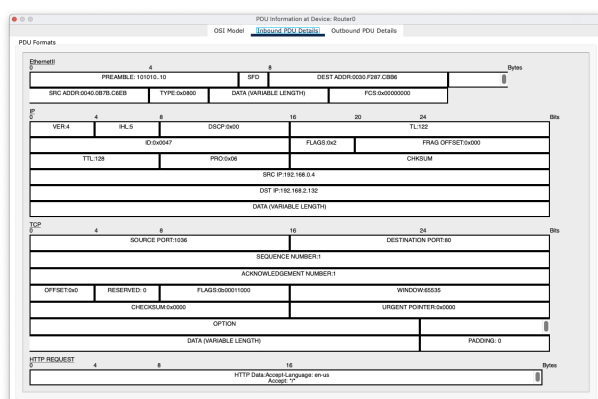


Figura 13: Empaquetat abans del router.

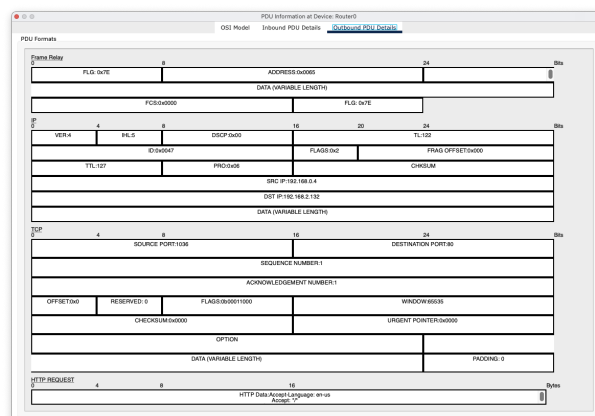


Figura 14: Empaquetat després del router.

La imatge final de la xarxa amb el servidor és aquesta:

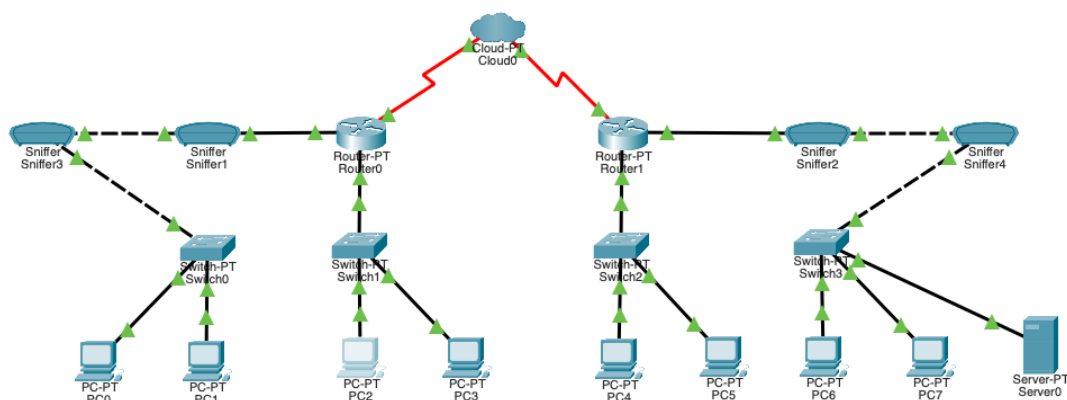


Figura 15: Xarxa final.

El tràfic generat l'hem anat mostrant al llarg de la resolució d'aquest exercici. Igualment, així llueix l'Sniffer4 després de la Simulation de la consulta de la pàgina web pel PC1 al servidor.

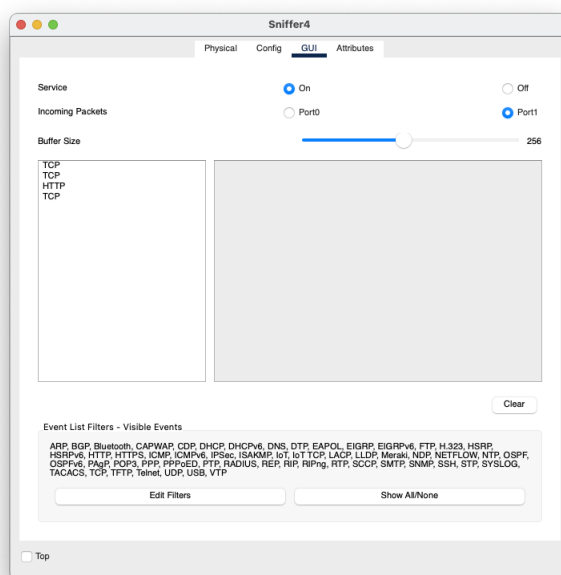


Figura 16: Tràfic capturat per l'Sniffer4 durant la simulació.

Finalment, apuntar que hem deixat el NAT activat. Pel que fa al Router0:

```

1 Router(config)#int Fa0/0
2 Router(config-if)#ip nat inside
3 Router(config-if)#exit
4 Router(config)#int Fa1/0
5 Router(config-if)#ip nat inside
6 Router(config-if)#exit
7 Router(config)#int Serial2/0

```

```

8 Router(config-if)#ip nat outside
9 Router(config-if)#exit
10 Router(config)#ip nat pool ROUTERO 161.116.95.3 161.116.95.100 netmask 255.255.255.0
11 Router(config)#access-list 1 permit 192.168.0.0 0.0.0.255
12 Router(config)#access-list 2 permit 192.168.1.0 0.0.0.255
13 Router(config)#ip nat inside source list 1 pool ROUTERO
14 Router(config)#ip nat inside source list 2 pool ROUTERO

```

Pel que fa al Router1:

```

1 Router(config)#int Fa0/0
2 Router(config-if)#ip nat inside
3 Router(config-if)#exit
4 Router(config)#int Fa1/0
5 Router(config-if)#ip nat inside
6 Router(config-if)#exit
7 Router(config-if)#int Serial2/0
8 Router(config-if)#ip nat outside
9 Router(config-if)#exit
10 Router(config)#ip nat pool ROUTER1 161.116.95.101 161.116.95.150 netmask 255.255.255.0
11 Router(config)# access-list 1 permit 192.168.2.0 0.0.0.127
12 Router(config)# access-list 2 permit 192.168.2.128 0.0.0.15
13 Router(config)#ip nat inside source list 1 pool ROUTER1
14 Router(config)#ip nat inside source list 2 pool ROUTER1

```

I amb això, ja hem acabat. ■

CONCLUSIONS

En aquesta pràctica hem explorat conceptes fonamentals de xarxes, com ara DNS, ICMP, TCP, així com el disseny i la configuració de topologies a través del simulador Cisco Packet Tracer. Els aspectes principals que hem treballat inclouen la captura i interpretació del trànsit real, així com la implementació de configuracions i protocols dins d'un entorn simulat.

Durant el procés, hem après a:

1. Capturar i analitzar tràfic de xarxa utilitzant eines com Wireshark, identificant diferents nivells de protocols i entenent el funcionament de capçaleres i missatges.
2. Configurar topologies complexes que incorporen subnetting variable (VLSM), assignació d'IP's estàtiques i dinàmiques, i mecanismes per evitar bucles amb protocols com STP.
3. Simular escenaris de connectivitat i serveis, com la creació d'un servidor web accessible des de diferents parts de la xarxa.

Els resultats obtinguts han demostrat que els protocols de xarxa en la comunicació i gestió del trànsit són essencials. Cisco Packet Tracer segueix sent una eina importantíssim per visualitzar i comprendre el flux de dades entre dispositius. També cal valorar el grau de concentració que es necessita en aquestes últimes pràctiques, ja que una mala assignació d'una sola IP, o a la taula d'enrutament, pot fer que tota la xarxa deixi de funcionar. La importància de la Així, la configuració dels components de xarxa ha de ser precisa per garantir la connectivitat i el rendiment.

Finalment, l'experiència ens ha permès consolidar coneixements teòrics i pràctics, preparant-nos per gestionar situacions reals, com sempre, amb esperit crític.